



14

Unit Testing in React

Although testing is an integral part of the software development process, developers and companies often pay surprisingly little attention to it in reality, especially to automated testing. In this chapter, we will try to understand why it is important to pay attention to testing and what advantages it gives. We will also explore the basics of unit testing in ReactJS, including general testing theory, tools, and methods, as well as specific aspects of testing ReactJS components.

In this chapter, we will cover the following topics:

- Testing in general
- Unit testing
- Testing ReactJS

Technical requirements

You can find the code files of this chapter on GitHub at

<https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter14>.

Testing in general

Software testing is a process aimed at identifying errors and verifying the functionality of a product to ensure its quality. Testing also allows developers and testers to assess the system's behavior under various conditions and to ensure that new changes have not led to regression, meaning they have not disrupted existing functionality.

The testing process includes a series of actions conducted to detect and identify any aspects that do not meet requirements or expectations. One example of such an action could be **manual testing**, where a developer or tester manually checks

the application. However, this approach is time-consuming and provides little guarantee that the application is secure and free of critical errors in operation.

To ensure a higher level of application reliability while saving time on testing, there are **automated tests**. They allow the functionality of the application to be verified without human intervention.

An automated test typically consists of a set of predefined tests and a software product, often referred to as a **runner**, which launches these tests and analyzes the results to determine the success or failure of each test. In addition to this, automated tests can be used to check performance, stability, security, availability, and compatibility, allowing you to write truly stable, large, and successful projects. That's why it's never a good idea to avoid tests; on the contrary, it's worth getting to know them better and trying to use them in all possible projects.

As developers, we are obviously more interested in automated testing than manual testing, so this chapter will focus on that. But before that, let's briefly look at the approaches to testing and the types of tests that exist.

Test types and approaches

Software testing can be classified by various criteria, including the level of testing and the objectives it pursues.

Typically, the following types of tests are distinguished:

- **Unit testing:** Testing individual modules or components of the program for correct operation. Unit tests are usually written and executed by developers to check specific functions or methods. Such tests are generally quick to write and can be executed quickly, but they do not test the final application for critical bugs, as the tested and stable components themselves may have problems when interacting with each other. An example of a unit test would be checking the functionality of a single function, React component, or Hook.
- **Integration testing:** Testing in which we check the interaction between various modules or system components. The goal is to detect defects in the interfaces and interactions between integrated components. This type of testing is usually conducted on the server side to ensure that all systems work smoothly together and that the business logic meets the specified requirements.

For example, an integration test would be one that checks that user registration works by making real calls to REST API endpoints and checking the returned data. Such a test depends less on the application's implementation and code and more on checking behavior and business logic.

- **End-to-end (E2E) testing:** Testing a complete and integrated software system to ensure that it meets specified requirements. E2E testing evaluates the program as a whole. This type of testing is the most reliable, as it completely abstracts from the application's implementation and checks the final behavior by interacting directly with the application itself. In the process of such testing, for example, in a web application, a real browser is launched in a special environment, in which a script performs real actions with the application like clicking buttons, filling out forms, and navigating through pages.

Although test types such as integration and E2E testing provide greater confidence in verifying the quality of an application, they come with drawbacks such as complexity and speed of test development, execution speed, and consequently, their costliness. Therefore, it is considered good practice to maintain a balance where preference is given to unit tests, as they are easier to maintain and faster to run. Then, all main business processes and logic are verified using integration tests, and E2E tests cover only the most critical business cases. This approach can be depicted in the form of a pyramid:

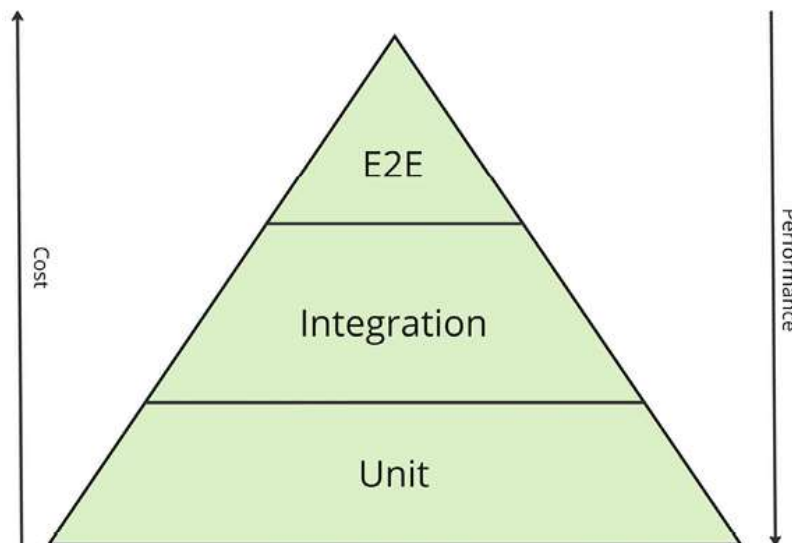


Figure 14.1: Test pyramid

The pyramid perfectly describes the approach we have discussed above. At its base lies **unit** testing, which should cover the application's source code as extensively as possible. It has the lowest cost of development and maintenance, as well as the highest test execution performance. In the middle are the **integration** tests, which are quite fast but more expensive to develop. At the very top, we have the **E2E** tests, which take the longest to execute and are the most expensive to develop, but they provide the maximum confidence in the quality of the product being tested.

Since integration and E2E tests abstract away from the implementation, and thus from the programming language or libraries used in the application, we are not going to cover those types of testing. Therefore, let's focus in more detail on unit testing.

Unit testing

We already know that unit testing is the process of verifying the correctness of individual “**units**” of code: namely, **functions** and **methods**. The goal of unit testing is to ensure that each separate unit performs its task correctly, which, in turn, increases confidence in the reliability of the entire application.

```
export function sum(a: number, b: number): number {  
  return a + b;  
}  
test('adds 1 + 2 to equal 3', () => {  
  expect(sum(1, 2)).toBe(3);  
});
```

The above represents the most basic and simplest test of a function that adds two values. The test code itself is a function that calls a special method, `expect`, which takes a value and then has a series of methods allowing for the checking and comparing of results.

Looking at this code, the first question that might come to mind is, is it really necessary to write another three lines of tests for such a simple three-line function? And why test such a function at all? I would answer with a definitive yes. It often

happens that a function can be covered by a test that is larger in volume than the function itself, and there is nothing wrong with that. Let's understand why.

Unit testing is most useful and effective when you are testing **pure functions**, which have no side effects and do not depend on external state. Conversely, unit testing is useless when the function being tested can change its behavior due to external factors or simply because that's how the function was designed. For example, functions for requesting data from the server, getting data from `localStorage`, or relying on global variables might return different results for the same input. From this, we can conclude that in an application development approach that requires code coverage with tests, you will automatically strive to write **testable code**, meaning more modular, independent, clean, and scalable code. This becomes especially noticeable on large projects. If tests were written from the start, such projects could continue to grow without the need for major refactoring or rewriting functionality from scratch. Also, in projects with tests, it is easier for newcomers to understand, as tests can serve as additional documentation for modules, reading which one can understand what the module is responsible for and what behavior it possesses.

For writing unit tests, there are entire concepts and methodologies. The main and most popular one is the traditional coverage of tests after code development. The advantage of this approach is the speed of development of the main functionality, as tests are usually dealt with later. Hence, the problem with this approach lies in delaying testing, which poses a risk of accumulating code that is not covered by tests. Later, when writing tests, it often becomes necessary to correct the main code, making it more modular and cleaner, which takes additional time.

There is also a methodology directly aimed at writing tests, called **test-driven development (TDD)**. This is a software development methodology in which tests are written before the code itself. The benefit of this approach is that your code will be immediately covered by tests, meaning it will be cleaner and more reliable. However, this approach may not be suitable for prototyping or for projects where requirements often change.

The choice between TDD and testing after development depends on many factors, including the team's culture, project requirements, and developers' preferences. It is important to understand that neither approach is a universal solution, and different choices may be justified in different situations. Most importantly, under-

stand the importance of testing and that one should avoid an approach in work where tests are not written at all as, in most cases, such code is doomed to be rewritten from scratch.

Now that we understand what unit tests are and their importance, let's take a closer look at them. Before writing tests, we should set up the environment in which we will run our tests.

Setting up the test environment

The most popular framework for writing and running unit tests is **Jest**. However, we will look at its more performant alternative, which is fully compatible with **Vite**, called **Vitest**. To install Vitest in your project, you need to execute this command:

```
npm install -D vitest
```

For basic operation, Vitest does not require any configuration, as it is fully compatible with the Vite configuration file.

Next, to get started, we need to create a file with the extension `*.test.ts`. The location of the file is not critical; the main thing is that the file is inside your project. Usually, test files are associated with the files of the functions being tested and are placed in the same directory; for example, for the `sum` function located in the `sum.ts` file, a file with a test named `sum.test.ts` is created and located in the same folder.

To run the tests, we need to add a launch script to the `package.json` file:

```
{
  "scripts": {
    "test": "vitest"
  }
}
```

Then, to call it, just execute the command in the terminal:

```
npm run test
```

This command will start the Vitest process, which will scan the project for files with the `.test` extension and then execute all the tests in each such file. Once all tests are completed, you will see the result in the terminal window, and then the process will wait for changes in the test files to rerun them. This is specifically designed as a mode for developing tests, where you do not need to constantly run the test command. For a one-time test run, you can add another command that will close the process upon completion of testing:

```
"test:run": "vitest run"
```

The `run` parameter is precisely for telling Vitest that you want to run the tests only once.

Vitest features

Let's now look at the main features of Vitest and the types of tests we can write. Let's start with a simple function, `squared`:

```
export const squared = (n: number) => n * n
```

This function returns the square of a number. Here is what the test for this function would look like:

```
import { expect, test } from 'vitest'
test('Squared', () => {
  expect(squared(2)).toBe(4)
  expect(squared(4)).toBe(16)
  expect(squared(25)).toBe(625)
})
```

The `test` and `expect` functions are part of the Vitest package. The test function takes the name of the test as its first argument and the test function itself as its second argument. The `expect` method serves as the basis for checking the expected result from the function being tested. Calling the `expect` method creates an object that contains a large number of methods, allowing for different ways to

check the execution result. In our example, we explicitly compare the result of executing the `squared` function with the expected value.

By running this test, in the terminal window, we will see the following message:

```
✓ test/basic.test.ts (1)
  ✓ Squared
Test Files  1 passed (1)
Tests      1 passed (1)
Start at   17:39:33
Duration   1.14s
```

To check that the tests are working correctly, let's change the expected value from 4 and see what result we get:

```
FAIL test/basic.test.ts > Squared
AssertionError: expected 4 to be 5 // Object.is equality
- Expected
+ Received
> eval test/basic.test.ts:13:22
   11|
   12| test('Squared', () => {
   13|   expect(squared(2)).toBe(5);
      |                       ^
   14|   expect(squared(4)).toBe(16);
   15|   expect(squared(25)).toBe(625);
```

—[1/1]—

```
Test Files  1 failed (1)
Tests      1 failed (1)
Start at    17:41:45
Duration    1.15s
```

When a test fails, we can see directly in the results where the error occurred, what result we received, and what we expected.

The `toBe` method is very useful for direct comparison of results, but what about objects and arrays? Let's consider this test example:

```
test('objects', () => {
  const obj1 = { a: 1 };
```

```
const obj2 = { a: 1 };  
expect(obj1).not.toBe(obj2);  
expect(obj1).toEqual(obj2);  
});
```

In this test, we created two identical objects, which will not be equal as variables. To expect the opposite assertion, we use the additional `.not.` key, which ultimately gives us the statement that the two variables are not equal to each other. If we still want to check that the objects have the same structure, there is a method called `toEqual`, which recursively compares objects. This method also works similarly with arrays.

For arrays, there are also additional methods that allow checking for the presence of an element, which is often very useful:

```
test('Array', () => {  
  expect(['1', '2', '3']).toContain('3');  
});
```

The `toContain` method can also work with strings and even DOM elements, checking for the presence of a class in `classList`.

The next big part of unit testing is working with functions. Vitest allows you to create **spyable fake functions**, which lets you check how and with what parameters this function was called. Let's look at an example function:

```
const selector = (onSelect: (value: string) => void) => {  
  onSelect('1');  
  onSelect('2');  
  onSelect('3');  
};
```

This function is created just for demonstration, but we can easily imagine some module or selector component that takes the `onSelect` callback, which will be called under some condition: in our case, three times in a row. Now let's see how we can test using observable functions:

```
test('selector', () => {  
  const onSelect = vi.fn();  
  selector(onSelect);  
  expect(onSelect).toHaveBeenCalledTimes(3);  
  expect(onSelect).toHaveBeenLastCalledWith('3');  
});
```

In the test, we created the `onSelect` function using the `vi` module from the `Vitest` package. This function now allows us to check how many times it was called and with what arguments. For this, we used the methods `toHaveBeenCalledTimes` and `toHaveBeenLastCalledWith`. There is also a method called `toHaveBeenCalledWith`, which can step-by-step check which arguments were used on each call of the observed function. In our case, the valid checks would be these three lines:

```
expect(onSelect).toHaveBeenCalledWith('1');  
expect(onSelect).toHaveBeenCalledWith('2');  
expect(onSelect).toHaveBeenCalledWith('3');
```

Vitest also allows you to spy a real function, for which you need to use the `vi.spyOn` method. However, for this, the function must be accessible from an object. Let's look at an example of spying on a real function:

```
test('spyOn', () => {  
  const cart = {  
    getProducts: () => 10,  
  };  
  const spy = vi.spyOn(cart, 'getProducts');  
  expect(cart.getProducts()).toBe(10);  
  expect(spy).toHaveBeenCalled();  
  expect(spy).toHaveReturnedWith(10);  
});
```

To create an observation for a function, we call `vi.spyOn` and pass it the object as the first argument and the name of the method as the second. Then, we can work with the original function and, later, make the necessary checks working with the `spy` variable. In the example above, you can also notice the new method `to-`

`HaveReturnedWith`, which allows you to check what the observed function returned.

Mocking

Moving on, I'd like to mention one of the most challenging parts of unit testing: namely, working with functions that have side effects or depend on external data or libraries. Earlier, I mentioned that testing is useless in functions with side effects, like calling something under the hood. Actually, that's not entirely true. In some cases, it's simply impossible to write a pure function, but that doesn't mean it can't be tested. To write tests for such functions, we can use **mocking**: namely, emulating external behavior or simply replacing the implementation of some modules or libraries.

An example could be a function that depends on the system time of the computer, or a function that returns data from a server. In such cases, we can apply a fake instruction that can change the current date of the computer specifically for this test, to have a clean result that is easier to test. Similarly, it is possible to create a fake implementation of a network request, which will ultimately be executed locally with the return of predetermined values. Let's discuss some of these scenarios in this section.

Consider the example of testing and using **timers**. In the testing environment, we can avoid waiting for timers and manually control them to more thoroughly test the behavior of the function. Let's look at an example:

```
function executeInMinute(func: () => void) {  
  setTimeout(func, 1000 * 60)  
}  
function executeEveryMinute(func: () => void) {  
  setInterval(func, 1000 * 60)  
}  
const mock = vi.fn(() => console.log('done'))
```

We created the `executeInMinute` and `executeEveryMinute` functions for delaying the call of a function by one minute and for cyclic execution every minute, respectively. We also created a mock function that we will subsequently spy on.

Here's what the test will look like:

```
describe('delayed execution', () => {
  beforeEach(() => {
    vi.useFakeTimers()
  })
  afterEach(() => {
    vi.restoreAllMocks()
  })
  it('should execute the function', () => {
    executeInMinute(mock)
    vi.runAllTimers()
    expect(mock).toHaveBeenCalledTimes(1)
  })
  it('should not execute the function', () => {
    executeInMinute(mock)
    vi.advanceTimersByTime(2)
    expect(mock).not.toHaveBeenCalled()
  })

  it('should execute every minute', () => {
    executeEveryMinute(mock)
    vi.advanceTimersToNextTimer()
    expect(mock).toHaveBeenCalledTimes(1)
    vi.advanceTimersToNextTimer()
    expect(mock).toHaveBeenCalledTimes(2)
  })
})
```

In this example, there's a lot to discuss but let's start with the fact that we didn't use the `test` function; instead, we used `describe` and `it`. The `describe` function allows us to create a test suite that can have its own context and lifecycle. In the test suite, we can set initial parameters or mock some behavior so that our test cases can reuse this context and these parameters later on. In our example, we use the `beforeEach` and `afterEach` methods, which set up fake timers before each test and then restore everything back to its original state after each test.

The `it` method is an alias for the `test` method and does not differ functionally from it. It's only there to make the test case more readable in the results. For example, `describe` with 'delayed execution' and `it` with 'should execute the function' in the results would look like this:


```
delayed execution > should execute the function
```

However, using `test`, we would see the result as:

```
delayed execution > if should execute the function
```

Now, let's look at the tests themselves. The first test uses the `executeInMinute` function, which, in reality, would call our observed method only after a minute, but in the test, we can control time. By using `vi.runAllTimers()`, we force the environment to start and skip all timers and immediately check the result. In the next test, we move time forward by 2 milliseconds using `vi.advanceTimersByTime(2)`, which already allows us to ensure that the original function will not be called.

Next, let's discuss the `executeEveryMinute` method, which should start a timer with a call to an argument every minute. In this case, we can step through each iteration of this timer using `advanceTimersToNextTimer`, giving us precise control over time without having to wait in real time.

When writing unit tests, we will often encounter that the function being tested will depend on some library or even a package.

Most often, you will encounter this in React Native, if a library or some method uses the device's native functions. In such a case, to write a test, we need to create a mock version of this logic that will be called during the test.

Let's consider a simple example where we imagine that we have a package that can interact with the device and get the current number of steps. To obtain the steps, we'll use the `getSteps` function:

```
export function getSteps() {  
  // SOME NATIVE LOGIC  
  return 100;  
}
```

As an example, the function itself will be very simple and will just return the value of `100`. However, in reality, such a function would interact with a smart-

phone API, which would be impossible to invoke within the scope of testing. Next, let's look at what we can do when writing a test:

```
import { beforeAll, describe, expect, it, vi } from 'vitest';
import { getSteps } from './ios-health-kit';
describe('IOS Health Kit', () => {
  beforeAll(() => {
    vi.mock('./ios-health-kit', () => ({
      getSteps: vi.fn().mockImplementation(() => 2000),
    }));
  });
  it('should return steps', () => {
    expect(getSteps()).toBe(2000);
    expect(getSteps).toHaveBeenCalled();
  });
});
```

The test and the entire example are quite primitive but they will give you an understanding of how mocking works. At the beginning of the file, we import our original package, `ios-health-kit`, then using the `beforeAll` method, we call `vi.mock`, passing it the path to the package as the first argument and a function that will return the implementation of the original file: namely, creating an object with the `getSteps` method as a fake function with an implementation that will return the value of `2000`. Then, in the test, we check that it indeed returns this value.

In this test, the `vi.mock` function creates a mock of the imported package and replaces the original import with it, which allows us to successfully test this functionality.

In fact, this example, in essence, does not test anything but merely demonstrates the possibility of mocking. In a real project, you will likely need to test functions that somewhere inside may use libraries that are important to mock. For this, it may be inconvenient to constantly manually write mocks before the actual test; to solve this, you can mock libraries and APIs at a global level. For this, you will need to create a configuration file or use `vi.stubGlobal`. I do not recommend diving so deeply right away without understanding and learning the basics, so let's move on.



More information about dependency mocking via configuration can be found at <https://vitest.dev/guide/mocking>.

The last but not least important example I'd like to discuss is mocking **network requests**. Almost any application you'll be developing will work with data that needs to be fetched from a server. For a unit test, this can be a problem, as it's important to test the unit abstracted from the external environment. Therefore, in unit tests, you should always mock server requests and provide the data necessary for the current test case. There's a library called `Mock Service Worker` for mocking server requests. It allows you to mock REST and GraphQL requests very flexibly. Let's look at an example:

```
import { http, HttpResponse } from 'msw';
import { setupServer } from 'msw/node';
import { describe, it, expect, beforeAll, afterEach, afterAll } from 'vitest';
const server = setupServer(
  http.get('https://api.github.com/users', () => {
    return HttpResponse.json({
      firstName: 'Mikhail',
      lastName: 'Sakhniuk',
    });
  })
);
describe('Mocked fetch', () => {
  beforeAll(() => server.listen());
  afterEach(() => server.resetHandlers());
  afterAll(() => server.close());
  it('should returns test data', async () => {
    const response = await fetch('https://api.github.com/users');
    expect(response.status).toBe(200);
    expect(response.statusText).toBe('OK');
    expect(await response.json()).toEqual({
      firstName: 'Mikhail',
      lastName: 'Sakhniuk',
    });
  });
});
```

In this test, we created a mock network request for the path `https://api.github.com/users`, which returns the data we need. For this, we

used the `setupServer` function from the `Mock Service Worker` package. Next, in the lifecycle methods, we set up the mock server to listen to server requests and then implemented a standard test where data is requested using the regular Fetch API. As you can see in the results, we can check the status code and the returned data.

With such a mocking approach, we truly have vast possibilities for testing different logic depending on the data returned from the server, status codes, errors, and so on.

In this section, we've introduced the basics of unit tests: namely, what they are and why we need to write them. We've learned how to set up the testing environment and write basic tests for our future projects. Next, let's move on to the main topic of our chapter, testing ReactJS components.

Testing ReactJS

We already know that unit testing involves checking small units, and most often, just functions, which perform some logic and return a result. To understand how testing in ReactJS works, the concept and idea remain the same. We know that at their core, React components are actually `createElement` functions that return a node, which, as a result of the `render` function, is displayed on the browser screen as HTML elements. In unit testing, we don't have a browser, but this is not a problem for us since we know that the render target in React can be almost anything. As you may have already guessed, in the unit tests of ReactJS components, we will be rendering components into a specially created **JSDOM** format, which is fully identical to the DOM, and the **React Testing Library** will help us with this.

This library contains a set of tools that allow rendering components, simulating events, and then checking the results in various ways.

Before we start, let's set up the environment for testing React components. To do this, in a fresh Vite project, execute this command:

```
npm install --save-dev \
  @testing-library/react \
  @testing-library/jest-dom \
  vitest \
  jsdom
```

This command will install all the dependencies we need. Next, we need to create a `tests/setup.ts` file to integrate Vitest and the React Testing Library:

```
import { expect, afterEach } from 'vitest';
import { cleanup } from '@testing-library/react';
import * as matchers from '@testing-library/jest-dom/matchers';
expect.extend(matchers);
afterEach(() => {
  cleanup();
});
```

Next, we need to update the `vite.config.ts` configuration file and add the following code there:

```
test: {
  globals: true,
  environment: "jsdom",
  setupFiles: "./tests/setup.ts",
},
```

These parameters tell Vitest to use an additional environment and execute our setup script before starting the tests.

The last step is to configure TypeScript typing, where we will specify that the `expect` function will now have additional methods to work with React components. To do this, we need to add the following code to the `src/vite-env.d.ts` file:

```
import type { TestingLibraryMatchers } from '@testing-library/jest-dom/matchers';
declare global {
  namespace jest {
    interface Matchers<R = void>
      extends TestingLibraryMatchers<typeof expect.stringContaining, R> {}
  }
}
```

This construction adds types for all the new methods provided by the React Testing Library. With this, the environment setup is complete, and we can proceed to writing tests.

First, let's consider the most basic check that a component has successfully rendered and is present in the document. For this, we'll create an `App` component that returns a title with the `Hello world` text:

```
export function App() {  
  return <h1>Hello world</h1>;  
}
```

The test for such a component would look like this:

```
import { render, screen } from "@testing-library/react";  
import { describe, it, expect } from "vitest";  
import { App } from "../App";  
describe("App", () => {  
  it("should be in document ", () => {  
    render(<App />);  
    expect(screen.getByText("Hello world")).toBeInTheDocument();  
  });  
});
```

The structure of the test itself is the same as before and is already familiar to us. The main thing to note is that at the beginning of the test, we render the component using the `render` function from `testing-library`, and after that, we can perform checks. To work with the rendering result, we use the `screen` module. It allows us to interact with our virtual DOM tree, and search for necessary elements in various ways.

We will cover the main ones later, but in this example, we used the `getByText` method, which queries for an element containing the text `"Hello World"`. To check whether this element is present in the document, we use the `toBeInTheDocument` method. This is how the output looks when you run the test:

```
✓ src/App.test.tsx (1)  
  ✓ App (1)  
    ✓ should be in document  
Test Files  1 passed (1)  
Tests      1 passed (1)  
Start at   14:19:01  
Duration   198ms
```

Now let's consider a more complex example, where we need to check that clicking a button adds a new `className` property to the component:

```
export function ClassCheck() {  
  const [clicked, setClicked] = useState(false);  
  return (  
    <button  
      className={clicked ? "active" : ""}  
      onClick={() => setClicked(true)}  
    >  
      Click me  
    </button>  
  );  
}
```

By clicking the button, we update the state, which updates the component and adds an `active` class to it. Now, let's write a test for this component:

```
describe("ClassCheck", () => {  
  it("should have class active when button was clicked", () => {  
    render(<ClassCheck />);  
    const button = screen.getByRole("button");  
    expect(button).not.toHaveClass("active");  
    fireEvent.click(button);  
    expect(button).toHaveClass("active");  
  });  
});
```

In this test, you first render the `ClassCheck` component, then we need to find the button element, and for this, we use the `screen` module with the `getByRole` method. This is the next method that allows query elements in the document, but it's important to understand that if there is more than one `button` in the document, this test will produce an error. Therefore, it's necessary to apply suitable query methods in different situations. Now that the button is accessible, we first ensure that the component does not contain the `active` class using the `toHaveClass` method with the `not` prefix.

To click on this button, the React Testing Library provides the `fireEvent` module, which allows generating click events. After clicking the button, we check that the

required class is present in the element.

With `fireEvent`, it's possible to generate all possible events such as click, drag, play, focus, blur, and many others. A very common event that is also important to test is the `change` event in an input element. Let's discuss this using the `Input` component as an example:

```
export function Input() {  
  return <input type="text" data-testid="userName" />;  
}
```

This component simply returns an `input` element, but in this example, I've also added a special attribute, `data-testid`. This is used for more convenient searching of elements in the document, as this attribute abstracts you from working with the content of the component or the role of the element. During the development of the project, you will often update your components, and the `data-testid` attribute will help you less frequently fix broken tests due to content updates or changes, say from `h1` to `h2` or `div` to a more semantic element.

Now let's write a test for this component:

```
describe("Input", () => {  
  it("should handle change event", () => {  
    render(<Input />);  
    const input = screen.getByTestId<HTMLInputElement>("userName");  
    fireEvent.change(input, { target: { value: "Mikhail" } });  
    expect(input.value).toBe("Mikhail");  
  });  
});
```

In this test, as usual, we render the component, and then we find our element using the more convenient method, `getByTestId`. Next, we simulate the `change` event on `input` using the `fireEvent.change` method, which takes the event object, and at the end of the test, we assert the entered value in the expected one. In this way, we can now test large forms with various logic for formatting, validation, and so on.

Just like testing components, the React Testing Library can also test Hooks. This allows testing only the custom logic and abstracting away from the components. Let's write a small `useCounter` Hook, which will return the current `counter` value and the `increment` and `decrement` functions:

```
export function useCounter(initialValue: number = 0) {  
  const [count, setCount] = useState(initialValue);  
  const increment = () => setCount((c) => c + 1);  
  const decrement = () => setCount((c) => c - 1);  
  return { count, increment, decrement };  
}
```

In order to test this Hook, instead of using the `render` function, the React Testing Library has a `renderHook` method. This is what a test of this Hook would look like:

```
test("useCounter", () => {  
  const { result } = renderHook(() => useCounter());  
  expect(result.current.count).toBe(0);  
  act(() => {  
    result.current.increment();  
  });  
  expect(result.current.count).toBe(1);  
  act(() => {  
    result.current.decrement();  
  });  
  expect(result.current.count).toBe(0);  
});
```

Initially, we render the Hook itself and check that the initial value is zero. The `renderHook` method returns the `result` object, through which we can read the data returned from the Hook. Next, we need to test the `increment` and `decrement` methods. To do this, it is not enough to simply call them, as Hooks are not pure functions by nature and contain a lot of logic under the hood. Therefore, we need to call those methods wrapped in the `act` method, which will synchronously wait for the method to execute and the Hook to re-render. After that, we can assert the expectations in the usual way. The output will look the same as we saw in the previous example, but let's now try to update the test to make failed results. Updating the first assertion from `.toBe(0)` to `.toBe(10)` will look like:

```
AssertionError: expected +0 to be 10 // Object.is equality
- Expected
+ Received
- 10
+ 0
> src/useCounter.test.ts:8:32
    6|   const { result } = renderHook(() => useCounter());
    7|
    8|   expect(result.current.count).toBe(10);
      |                                   ^
    9|
   10|   act(() => {
```

```
Test Files  1 failed (1)
Tests       1 failed (1)
Start at    14:24:06
Duration    200ms
```

You will notice how Vitest highlights the part of the code where we got a failed assertion.

In this section, we have learned how we can test components and Hooks using the React Testing Library.

Summary

In this chapter, we explored the broad and extensive topic of testing. We became acquainted with the concept itself, testing types, and various approaches. Then, we delved into unit testing and learned what it is, and what possibilities this type of testing offers. After that, we learned how to set up the environment and write tests for regular functions and logic. At the end of the chapter, we examined the basic capabilities of testing React components and Hooks.

With this chapter, we conclude our acquaintance with the amazing ReactJS library and will next dive deeper into the React ecosystem with the incredible opportunity to create mobile applications based on React Native.

Join us on Discord!

Read this book alongside other users and the authors themselves. Ask questions, provide solutions to other readers, chat with the authors, and more. Scan the QR

code or visit the link to join the community.

<https://packt.link/ReactAndReactNative5e>

